120. Triangle

PLATFORM	DIFFICULTY	DATE
Platform: LeetCode	MEDIUM	Solved: October 15, 2026

Problem Summary

Given a triangle (list of lists), return the minimum path sum from top to bottom. From cell `(i, j)` you may move only to `(i+1, j)` or `(i+1, j+1)`.

Approach

Greedy (pick the smaller neighbor) fails because a locally smaller choice can block a cheaper path below. Use DP from the bottom: `dp[j]` holds the min path sum from cell `j` of the current row to the last row. Recurrence: `f(i, j) = triangle[i][j] + min(f(i+1, j), f(i+1, j+1))`. Row `i` only needs row `i+1`, so one 1D array is updated in place and `dp[0]` is the answer.

Key Insights

- Greedy is wrong: 1 -> 2 -> 8 = 11, but 1 -> 3 -> 1 = 5.
- Bottom-up means every cell already knows the best result below it.
- Row `i` has `i+1` elements, which bounds the inner loop.
- Left-to-right update is safe: `dp[j]` reads `dp[j]` and `dp[j+1]`, not yet overwritten.

Observations

- Sorting rows breaks the index adjacency rule.
- Last row needs no computation; its values are the base case.
- A full 2D table also works but costs $O(n^2)$ space.
- No visited array needed; the triangle shape bounds the loops.

Approaches

- **Brute Force / Recursion $O(2^n)$ time:** From the top, try both moves at every row and take the min. Same subproblems are recomputed.
- **Top-Down Memoization $O(n^2)$ space:** Cache `f(i, j)` in a 2D table so each cell is solved once.
- **Optimal / Bottom-Up 1D DP O(n) space:** Start from the last row and fold each row into one `dp` array going upward. $O(n^2)$ time.

Algo

1. Let `n` = number of rows; create `dp[n]`.
2. Copy the last row into dp (base case).
3. For `i` from `n-2` down to `0`:
4. For `j` from `0` to `i`: `dp[j] = triangle[i][j] + min(dp[j], dp[j+1])`.
5. Return `dp[0]`.

Dry Run

Step	Computation	dp after
init	copy last row	[9, 8, 1]
i=1, j=0	2 + min(9, 8) = 10	[10, 8, 1]
i=1, j=1	3 + min(8, 1) = 4	[10, 4, 1]
i=0, j=0	1 + min(10, 4) = 5	[5, 4, 1]

Triangle: `[[1],[2,3],[9,8,1]]`. Answer = `dp[0]` = 5.

Section

Time Complexity

$O(n^2)$, n = number of rows (each cell visited once).

Space Complexity

O(n) — the 1D `dp` array.

Points to Remember

- "Min/max path with overlapping choices" signals DP, not greedy.
- Think bottom-up when each state depends on the next level.
- Define `f(i, j)` first, then the recurrence, then the base case.
- Roll the DP array in place when only the previous row is needed.
- Same pattern: Minimum Falling Path Sum, Unique Paths, Min Path Sum (grid).

Solution Code — Java

```
class Solution {
    public int minimumTotal(List<List<Integer>> triangle) {
        int n = triangle.size();
        int[] dp = new int[n];

        for (int j = 0; j < n; j++) {
            dp[j] = triangle.get(n - 1).get(j);
        }

        for (int i = n - 2; i >= 0; i--) {
            for (int j = 0; j <= i; j++) {
                dp[j] = triangle.get(i).get(j)
                      + Math.min(dp[j], dp[j + 1]);
            }
        }
        return dp[0];
    }
}
```